

Strings in JavaScript

In JavaScript, strings can refer to both **primitive data types** and **objects** depending on how they are created.

1. String as a Primitive Data Type

When you create a string using a **string literal**, it is considered a primitive data type.

Example:

```
let fname = "Yogesh";  
console.log(typeof fname); // Output: string
```

In this case, `fname` is a string literal, and `typeof fname` returns `"string"`, indicating that it is a primitive string.

2. String as an Object

When you create a string using the **`new String()`** constructor, it is treated as an object. Even though it behaves similarly to a string, it is technically an instance of the `String` object.

Example:

```
let fname = new String("Yogesh");  
console.log(typeof fname); // Output: object
```

Here, `fname` is created using the `new String()` constructor, which results in an object. Thus, `typeof fname` returns `"object"`.

3. Strings are Immutable

In JavaScript, strings are **immutable**, meaning once a string is created, its value cannot be changed. Any operation that seems to modify the string actually creates a new string.

Example:

```
let name = "Yogesh";  
console.log(name); // Output: Yogesh  
  
// Attempting to modify the string  
name = "Yogesh Pandian";  
console.log(name); // Output: Yogesh Pandian
```

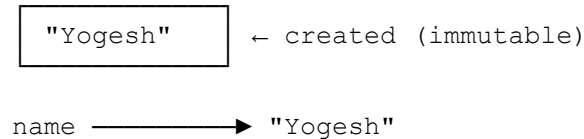
In the example above, the variable `name` initially holds the value `"Yogesh"`. When we reassign it to `"Yogesh Pandian"`, it does not modify the original string. Instead, it creates a new string and assigns it to the variable `name`. Strings in JavaScript do not change in place.

Step-by-step Memory Representation:

Step 1:

```
let name = "Yogesh";
```

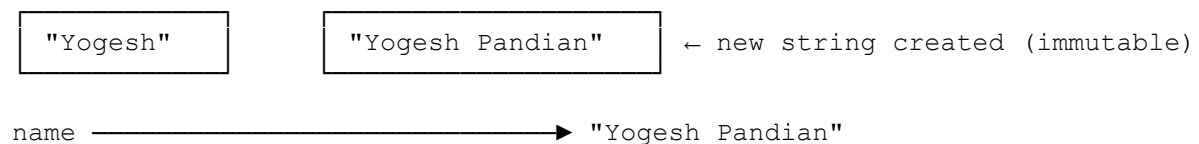
Memory:



Step 2:

```
name = "Yogesh Pandian";
```

Memory:



The original "Yogesh" string **did not change**.

The variable `name` now points to a **completely new string**.

Why It's Called Immutable:

Because even if you tried this:

```
let name = "Yogesh";
name[0] = "Z";
console.log(name); // Still "Yogesh"
```

...it wouldn't work. You **can't change individual characters** of a string.

4. Important Notes on Immutability

- Strings in JavaScript are immutable, meaning that the original value of a string can't be altered.
- If you attempt to modify a string (e.g., by changing a character in the string), it will create a new string instead of modifying the original one.
- You cannot directly update individual characters in a string because they are not changeable.

Example of Immutability:

```
let name = "Yogesh";  
name[0] = "P"; // This has no effect  
console.log(name); // Output: Yogesh
```

In the above code, we try to change the first character of the string from "Y" to "P". Since strings are immutable, the original string remains unchanged.

5. String Concatenation

To update or change the string, you would need to concatenate new values or assign a new string to the variable.

Example:

```
let name = "Yogesh";  
name = name + " Pandian";  
console.log(name); // Output: Yogesh Pandian
```

In this case, we concatenate the string " Pandian" to the original string "Yogesh", which results in the new string "Yogesh Pandian".

Summary

- **String literal ("value")** creates a primitive string.
 - **new String("value")** creates a string object.
 - Strings in JavaScript are immutable, meaning their values cannot be directly modified.
 - Reassigning a string variable creates a new string rather than modifying the original string.
-

String Functions in JavaScript

Consider the following string:

```
let message = "My name is Yogesh Pandian";
```

1. Length

The `length` property returns the number of characters in the string, including spaces.

```
console.log(message.length); // Output: 23
```

To get a specific character, you can access it using indexing:

```
console.log(message[4]); // Output: "n" (the character at index 4)
```

2. Includes()

The `includes()` method checks if a substring is present in the string.

```
console.log(message.includes("Yogesh")); // Output: true
```

3. StartsWith()

The `startsWith()` method checks if the string starts with a given substring.

```
console.log(message.startsWith("My")); // Output: true
```

4. EndsWith()

The `endsWith()` method checks if the string ends with a given substring.

```
console.log(message.endsWith("Pandian")); // Output: true
```

5. Split()

The `split()` method splits the string into an array of substrings based on a separator.

```
let arr = message.split(" "); // Split by space
console.log(arr); // Output: ["My", "name", "is", "Yogesh", "Pandian"]

// To get a specific string:
console.log(arr[4]); // Output: "Pandian"
```

6. toUpperCase()

The `toUpperCase()` method converts the entire string to uppercase.

```
console.log(message.toUpperCase()); // Output: "MY NAME IS YOGESH PANDIAN"
```

7. toLowerCase()

The `toLowerCase()` method converts the entire string to lowercase.

```
console.log(message.toLowerCase()); // Output: "my name is yogesh pandian"
```

8. trim()

The `trim()` method removes whitespace from both ends of the string.

```
let spacedMessage = "  Hello World!  ";  
console.log(spacedMessage.trim()); // Output: "Hello World!"
```

9. replace()

The `replace()` method replaces part of the string with another string. Note that it returns a new string and does not modify the original string (because strings are immutable).

```
console.log(message.replace("Pandian", "SDET")); // Output: "My name is Yogesh SDET"
```

```
// The original string remains unchanged:  
console.log(message); // Output: "My name is Yogesh Pandian"
```

```
// If you want the updated string, assign it to a new variable:  
let newMessage = message.replace("Pandian", "SDET");  
console.log(newMessage); // Output: "My name is Yogesh SDET"
```

10. Extracting and Validating a Number from a String

If your string contains a number and you want to extract and validate it:

```
let messageWithNum = "My height is 180 cm";  
  
// Split the string and get the 3rd element (the number as a string)  
let numberString = messageWithNum.split(" ")[3]; // "180"  
console.log(typeof numberString); // Output: "string"  
  
// Convert it to a number using `parseInt()`  
let number = parseInt(numberString);  
console.log(typeof number); // Output: "number"  
  
// Now you can validate the number:  
if (number === 180) {  
    console.log("Pass");  
} else {  
    console.log("Fail");  
}
```

11. Escape Sequences

In strings, you can use escape sequences like `\n` for a new line, `\t` for a tab, and more.

- **New line:** `\n`
- **Tab:** `\t`
- **Backslash:** `\\`

```
let exampleString = "Hello\nWorld!\tThis is a test.";
console.log(exampleString);
// Output:
// Hello
// World!    This is a test.
```

12. Template Literals (Optional)

Using template literals allows you to easily embed variables into strings. Template literals use backticks (```) instead of quotes.

```
let name = "Yogesh";
let profession = "SDET";
let message = `My name is ${name} and I am a ${profession}.`;
console.log(message); // Output: "My name is Yogesh and I am a SDET."
```

Conclusion:

These are some of the most commonly used string functions in JavaScript. Strings in JavaScript are **immutable**, meaning any operation that appears to change the string will actually create a new string.
